MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY — SOFTWARE ENGINEERING

OMNICAST | BROADCAST INTELLIGENCE PLATFORM

FINAL PROJECT DELIVERABLE REPORT

COURSE: PRN232 — BUILDING CROSS-PLATFORM BACK-END APPLICATION

OMNICAST: DIGITAL BROADCAST SCHEDULING & DELIVERY SYSTEM

Enterprise Media & Broadcast Intelligence Network

NestJS gRPC Microservices, Next.js 15 Web Portal, 24/7 EPG Conflict Detection Algorithm and AI Curation

Course Instructor:	Department of Software Engineering
Academic Semester:	Fall 2026 (FA26)
Author Team:	OmniCast Engineering Group
GitHub Repository:	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Live Deployment:	https://omnicast-fe.vercel.app (API: /api/v1)
Scale & Verification:	**142,897 LOC | 482/482 Tests PASS (100%)**

VIETNAM — OCTOBER 2026

Building a Cross-Platform Back-End Application: The OmniCast Digital Broadcast Scheduling and Delivery System

OmniCast Engineering Team — PRN232 (Fa26)

Department of Software Engineering, FPT University
Source Code: https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Live System: https://omnicast-fe.vercel.app

***Abstract*— OmniCast is a multi-platform digital broadcast scheduling and delivery system built for the PRN232 course. It comprises a NestJS back end with 19 business modules and 140 REST endpoints, a Next.js 15 web portal with 33 routes, and a gRPC audit-logging microservice. Three principal contributions are presented: a broadcast-schedule conflict-detection algorithm implemented as a single raw SQL query that computes the effective end time of an existing event; a 24/7 EPG filler whose seeded selection guarantees that every channel-day grid is both complete and reproducible; and an AI curation pipeline that issues a single large language model call, validates the output against a Zod schema, offers a heuristic fallback path and records full version provenance. The database schema comprises 21 Prisma models on PostgreSQL. The complete system has been deployed on Vercel and validated by 482 automated tests inside a five-job continuous-integration pipeline, all of which pass.**

***Index Terms*—broadcast scheduling, EPG, NestJS, Prisma, Next.js, gRPC, RBAC, Flutter, AI curation, conflict detection, offline-first, CI/CD.**

I. Introduction

In recent years, the way Vietnamese audiences consume television has changed dramatically. Viewers no longer sit in front of a screen at a fixed broadcast slot; they browse on-demand, miss the schedules of channels they follow, and expect content recommendations tailored to their interests. Broadcasters and digital-content providers, in turn, need a single, reliable and scalable scheduling system that serves several delivery surfaces at once: the web, mobile applications and live-streaming devices.

This paper presents **OmniCast** — a multi-platform digital broadcast scheduling and delivery system built for the course **PRN232 — Building Cross-Platform Back-End Application**. The system comprises a high-performance NestJS back end that serves both a Next.js 15 web portal and a Flutter mobile application, with the core business logic concentrated on three capabilities: an electronic programme guide (EPG) that is never empty, a broadcast-schedule conflict-detection mechanism, and an AI editorial assistant with content-compliance validation.

A. Motivation and Problem Statement

Three technical problems dominate the system's design, each derived from concrete project requirements.

1) **Sparse schedules.** A real broadcast channel originates only a handful of productions per day. Building a 24-hour EPG grid directly from the event table leaves most of the grid empty and renders the day filter in the user interface meaningless. The system therefore requires an automatic filler that guarantees every channel-day grid is fully populated with valid content.
2) **Schedule conflicts.** If two programmes on the same channel overlap, viewers cannot tell which one is actually on air. The validation must detect all three overlap forms (head, tail, full containment) and must execute as a single database query to avoid race conditions.
3) **Manual content moderation.** The editorial workflow (writing descriptions, tagging, categorising) is manual and time-consuming. Automating it with a large language model reduces the load, but requires a fallback path when no API key is present, and the output must be schema-validated before it is persisted.

B. Contributions

This paper makes the following contributions.

1) A layered **NestJS + Prisma + PostgreSQL** architecture with 19 business modules, 140 REST endpoints and auto-generated OpenAPI documentation, protected by a four-role RBAC layer.
2) A broadcast-conflict detection algorithm implemented as a single raw SQL query that computes the effective end time with `COALESCE(endedAt, scheduledAt + duration)`.
3) A 24/7 EPG filler with seeded pseudo-random selection keyed on the (date, channel) pair, guaranteeing that the same day always renders the same schedule.
4) An AI editorial pipeline that issues a single consolidated model call (`gpt-4o`), validates the result against a Zod schema, and provides retry plus heuristic-fallback paths.

5) An audit-logging microservice communicating over gRPC, deployed independently of the main API process.
6) A Next.js 15 web portal with 33 routes and an AI Curator studio built on the Next.js App Router architecture.
7) A five-job GitHub Actions CI/CD pipeline with an automated quality gate, covering 482 automated tests across three suites.

C. Scope

This paper concentrates on the back end and the web portal, which are the two PRN232 deliverables. The Flutter mobile application is presented separately in the companion PRM393 paper. Infrastructure deployment, environment-variable configuration and mobile release procedures are covered in the accompanying deployment documentation and are only summarised here at the architectural level.

II. Related Work

This paper positions OmniCast against three main lines of work: back-end application frameworks, data-access protocols, and broadcast scheduling systems.

A. Back-end Application Frameworks

NestJS [1] adopts a decorator-driven dependency-injection model and a hierarchical module model, allowing each business domain (channels, programmes, users) to be packaged as an independent module with its own lifecycle. Compared with plain Express, the framework provides out-of-the-box URI versioning (`VersioningType.URI`), a `ValidationPipe` in `forbidNonWhitelisted` mode, an `ExceptionFilter` and an `Interceptor` — the four components the broadcasting problem demands so that every endpoint returns an identical response envelope and every input is validated strictly. Prisma [2] serves as the data-access layer: the Prisma schema is the single source of truth and `prisma migrate` generates version-controlled SQL, so schema drift is tracked alongside the source code.

Two alternatives deserve comparison: FastAPI (Python) and ASP.NET Core (C#). FastAPI produces excellent OpenAPI documentation but the project has a fixed language budget; ASP.NET Core has a more mature ecosystem but also demands a language switch. NestJS allows the TypeScript toolchain to be reused with the Next.js portal [3], so a shared data type or an API contract change can propagate across both deliverables without a language boundary.

B. Data-Access Protocols

Most legacy broadcasting systems expose their programme guide to mobile applications through OData [10], a REST extension with embedded query syntax ($filter, $orderby, $expand). OData is well suited when clients need flexible queries over a large catalogue without deploying server-side filtering logic. OmniCast takes the opposite approach: EPG queries have a fixed shape (by day, by channel, by time window), so plain REST endpoints with normalised query parameters [7] are sufficient and simpler for both clients.

Compared with GraphQL [9], plain REST lowers the client-side cost because no additional runtime layer is needed on mobile, and it is easier to cache at the HTTP layer and to exercise from a command line. The trade-off is that every endpoint must be designed explicitly; in exchange, the entire API is documented with OpenAPI 3 [8] using `@ApiOperation`, `@ApiResponse` and `@ApiQuery` decorators, so no endpoint is left undocumented.

C. Broadcast Scheduling and AI Assistance

Commercial scheduling systems (such as EBU Radio-MEP and FTA EPG feeds) guarantee schedule integrity by applying relative-time conflict detection and substituting overrun programmes during live transmission. This is precisely the model OmniCast reproduces at a smaller scale: the slot of a programme is an interval $[t_0, t_1]$ on the timeline of a channel, and two overlapping intervals are invalid.

On the editorial-assistance side, commercial EPG systems have recently begun using large language models to generate descriptions and content tags [21]. OmniCast's distinguishing feature is *output quality control*: every AI-generated report is validated against a Zod schema [22] before being written to the database and is stamped with the prompt version and the model version, so administrators can compare quality across prompt revisions.

D. Gap Addressed by OmniCast

In summary, the gap this project fills is the absence of a complete, automatically verifiable broadcast-scheduling system at classroom scale: schedule-integrity guarantees, a full user experience across web and mobile, quality-controlled AI assistance, and the whole thing validated by 482 automated tests inside a continuous-integration pipeline.

III. System Architecture

A. Layered Overview

The system is organised into four layers, each communicating only with the layer immediately below it. The presentation layer comprises the Next.js 15 web portal and the Flutter mobile application; the API layer is a single NestJS process with 19 business modules; the real-time layer handles live broadcasting, chat and WebRTC signalling; and the data layer consists of PostgreSQL, the MediaMTX streaming server, Cloudinary storage and external APIs.

The notable design decision is a *single API process serving both clients*. The boundary between the web and mobile clients exists only at the transport layer, never at the business layer. A business rule is therefore written exactly once: for instance, the rule that "two programmes on the same channel must not overlap" is enforced in `ProgramsService.checkScheduleConflict` and behaves identically whether the schedule is created from the web administration portal or from the mobile application.

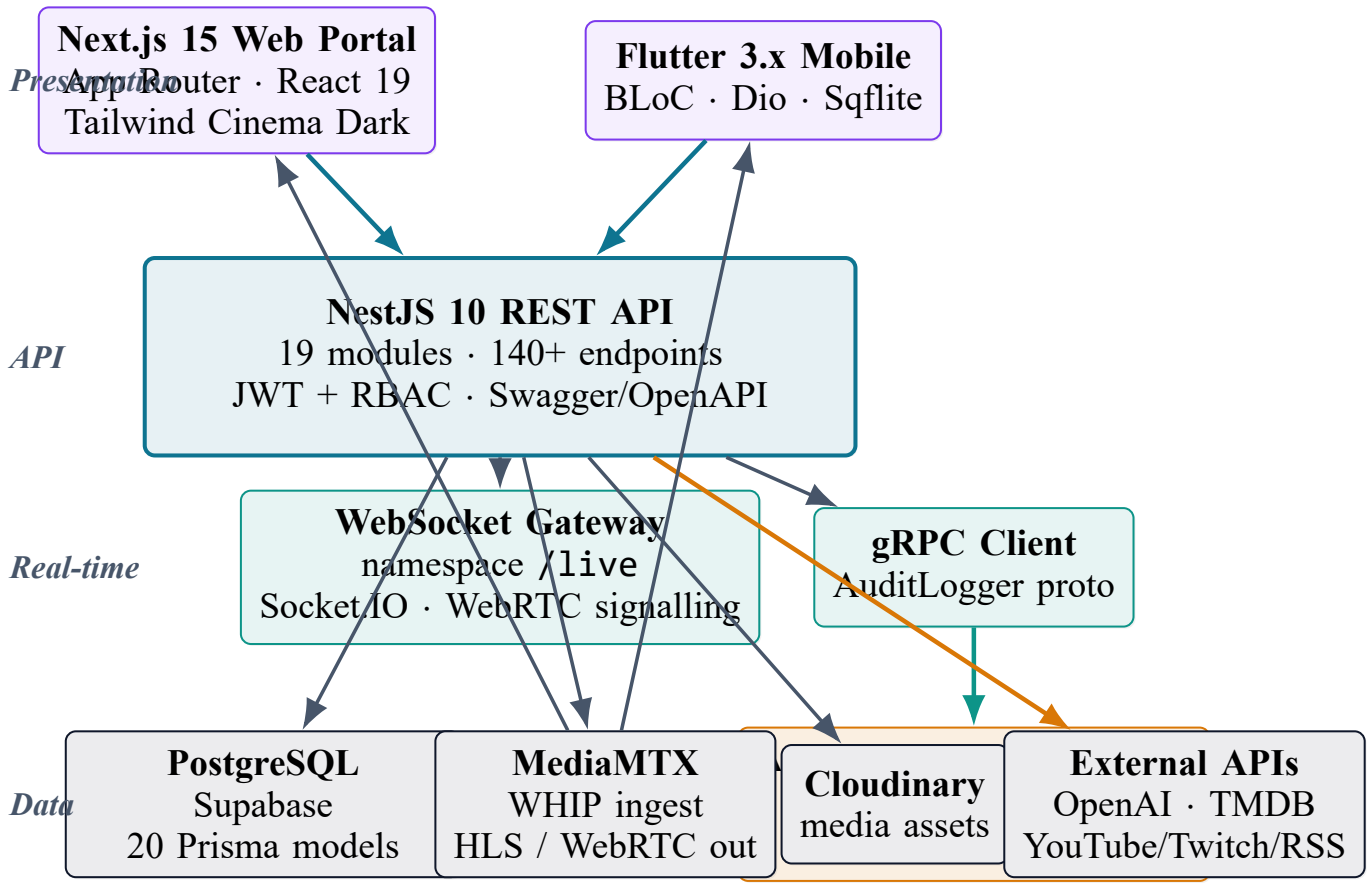


Figure 1. Overall layered architecture of OmniCast. Both the web portal and the mobile application consume the same REST API; audit logging is separated into a dedicated gRPC microservice.

B. NestJS Module Structure

`AppModule` registers 19 business modules alongside the infrastructure modules. Each module owns one business domain and exports the services it needs; controllers only call services and hold no business logic. Table I lists the actual back-end source layout.

C. Protocol and Transport Integration

The back end serves four independent protocols, each serving a distinct group of needs.

- **REST/JSON over HTTP/1.1.** Paths carry the `/api/v1` prefix and use `VersioningType.URI`. A middleware in `main.ts` automatically rewrites any `/api/*` request to `/api/v1/*`, so older clients never receive a 404 when the API version is upgraded.
- **WebSocket.** The realtime gateway uses Socket.IO on namespace `/live` and exposes 12 events: event and channel subscribe/unsubscribe, live chat, reactions, broadcaster messages, message deletion, and the WebRTC handshake (offer/answer/ICE candidate).
- **gRPC.** The audit-logging microservice receives `LogAudit`, `GetLogsByUser`, `GetLogsByAction`, `GetLogsByEntity` and `GetLogStats` requests over `audit_logger.proto`.

Table I
BACK-END NESTJS MODULE STRUCTURE.

Module	Primary responsibility	Controller
`auth`	Login, JWT, refresh, Google sign-in	`auth`
`channels`	Broadcast channels, catalogue	`channels`
`programs`	Programmes, EPG, live studio	`programs`
`search`	Cross-entity search	`search`
`ai-curator`	AI curation, statistics	`ai-curator`
`audit-logger`	Audit logging (gRPC)	`audit-logger`
`watchlist`	Watchlist	`watchlist`
`users`	User management	`users`
`admin`	Admin dashboard	`admin`
`billing`	Plans, payments	`billing`
`notifications`	Notifications, FCM push	`notifications`
`newsletter`	Newsletter subscription	`newsletter`
`websocket`	Realtime gateway	—
`cloudinary`	Media storage	`cloudinary`
`prisma`	Database access	—
`health`	Health probe	`health`
`common`	Filters, interceptors, Swagger	—

- **WHIP/HLS/WebRTC.** MediaMTX ingests over WHIP and redistributes over HLS (LL-HLS, fMP4) and WHEP, configured in `backend/mediamtx/mediamtx.yml`.

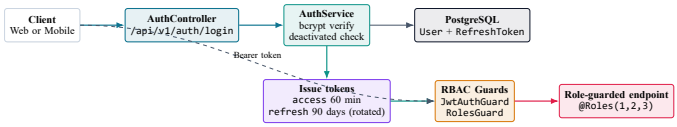


Figure 2. Authentication and authorisation flow. The access token accompanies every request; `JwtAuthGuard` validates the signature and `RolesGuard` compares the role against the `@Roles` metadata.

D. Database Connection and Audit Logging

The database is Supabase-managed PostgreSQL. One operational detail is worth noting: Supabase exposes two ports, `5432` for direct connections and `6543` for the transaction pooler. The `optimizeDatabaseUrl` helper in `PrismaService` automatically switches to `6543` and sets a suitable `connection_limit` when it detects the pooler, avoiding connection exhaustion when a serverless environment starts several instances concurrently.

Why a separate audit-logging microservice?

Audit logging has two properties the rest of the system does not share: a high write volume and a tolerance for delay. Writing straight into PostgreSQL would make every administrative operation wait on an extra write. Moving it into a gRPC microservice turns auditing into a non-blocking fire-and-forget operation: `AuditLoggerService` issues the RPC inside a non-awaited promise, while the microservice writes to the database and maintains index tables supporting queries by user, action, entity and aggregate statistics.

E. External Integrations

Four external integrations are supported, all of which degrade gracefully when the API key is absent: YouTube Data API v3 (channel schedules), Twitch Helix (channel schedules), TMDB (film metadata enrichment) and OpenAI (content curation). This mechanism allows the whole system to run at full capability in a development environment without any third-party credential.

IV. Back-End Implementation

A. JWT Authentication and Role-Based Access Control

The system authenticates with an access/refresh token pair. At login the password is verified with `bcrypt.compare` at 12 rounds (`BCRYPT_ROUNDS=12`); at registration it is hashed with `bcrypt.hash`. `AuthService` issues a 60-minute access token and a 90-day refresh token stored in the `RefreshToken` table. Token rotation is enforced: on every refresh the old token is revoked and a new one is issued, so a leaked token cannot be reused.

Two guards are implemented in `backend/src/auth/guards`. The `JwtAuthGuard` uses the Passport-JWT strategy and skips handlers marked `@Public()`. The `RolesGuard` reads `@Roles` metadata through the `Reflector`; a handler that declares no roles admits every authenticated user. A notable detail is that the guard normalises roles to string names before comparing, so both the name form (`'STAFF'`) and the legacy numeric form (`'2'`) are accepted. The four roles are declared in the `UserRole` enum and ranked in the portal's `role-config.ts`.

Table II
THE FOUR USER ROLES AND THEIR PERMISSION SCOPE.

Code	Value	Description
`GUEST`	0	Unauthenticated guest – public schedule and trailers only
`VIEWER`	1	Signed-in viewer – watchlist, comments, likes, watch history
`STAFF`	2	Content editor – manage channels, schedule, run AI Curator
`ADMIN`	3	Administrator – all STAFF rights plus user and audit-log management

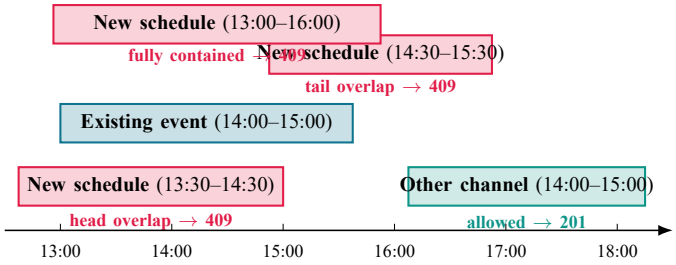


Figure 3. The three detected overlap forms: head overlap, tail overlap and full containment. A schedule on a different channel causes no conflict.

To enforce least privilege, sensitive operations are guarded at the controller level rather than in the user interface. For example, all eleven write endpoints in the `ai-curator` group require `@Roles('STAFF','ADMIN')`, and every user-management or audit-log endpoint requires `@Roles('ADMIN')`.

B. Broadcast Schedule Conflict Detection

The conflict-detection algorithm is the business core of the system. Each programme occupies a time interval $[t_0,\ t_0+d]$ on the timeline of one channel. Two programmes collide if and only if the new interval starts before the existing one ends and ends after the existing one starts.

The hardest part is computing the *effective* end time of an existing event. The `endedAt` column can still be `NULL` for a `SCHEDULED` event, meaning the event has not actually finished. A naive Prisma `where` clause would skip such events, producing false negatives that allow conflicting schedules. The `checkScheduleConflict` helper therefore uses raw SQL to compute the effective end time with a `COALESCE`/`MAX` expression over `endedAt` and `scheduledAt + duration`.

```
SELECT id, title, "scheduledAt" AS scheduled_at,
    "duration"
FROM "LiveEvent"
WHERE "channelId" = $1::uuid
  AND status IN ('SCHEDULED', 'LIVE')
  AND "scheduledAt" < $2                        -- moi
    bat dau truoc khi cu ket thuc
  AND COALESCE(
        "endedAt",
```

```
        "scheduledAt" + (
          CASE
            WHEN "duration" IS NULL THEN ($3 || ' minutes')::interval
            WHEN "duration" > 1440  THEN ("duration" || ' seconds')::interval
            ELSE ("duration" || ' minutes')::interval
          END)
        ) > $4                                  -- moi ket thuc sau khi cu bat dau
```

Listing 1. Conflict detection in a single raw SQL query. The `excludeEventId` clause lets the same helper be reused when updating an existing event.

The query only considers events in the `SCHEDULED` or `LIVE` states; `ENDED` and `CANCELLED` events cannot cause a conflict. The `CASE` expression also absorbs a unit inconsistency: the ingest layer writes `duration` in *seconds* while the EPG logic works in *minutes*, so values above 1440 are interpreted as seconds and converted, and everything else is treated as minutes. On a detected collision the service throws a `ConflictException` carrying HTTP status 409 together with the list of conflicting events, so the user interface can name the programme that caused the rejection.

C. 24/7 EPG Filler Mechanism

Because channels originate only a handful of productions per day, the 24-hour EPG grid is materialised by `EpgScheduleFillerService`. The service guarantees five properties demanded by the design.

1) **No empty days.** Every channel-day grid covers the full 24 hours, so the day filter in the user interface always has content to render.
2) **Genre-aware selection.** Recordings whose `category` matches the channel category are preferred; the full library is used only when nothing matches.
3) **Episode splitting.** Recordings longer than `LONG_PROGRAM_MINUTES` are split into chunks of `EPISODE_MINUTES` so the grid exposes "Episode 1 / Episode 2 / ..." entries instead of one massive block.
4) **Limited replay.** The same recording or episode may appear at most `MAX_REPEATS_PER_ITEM_PER_DAY` times in a single day, so viewers never see an episode loop endlessly.
5) **Reproducible randomness.** Selection uses a seeded PRNG keyed on the (date, channel) pair, so the same day always renders the same schedule while adjacent days surface different content.

The technically important point is that the service performs *no I/O*: it is pure logic that receives the recording list and the real programme list and returns the filled grid. As a result the 18 unit tests in `epg-filler.spec.ts` run without a database, and the whole scheduling layer becomes a pure function verifiable by properties.

D. Content Ingestion and Enrichment

The ingestion layer collects schedules from YouTube, Twitch and RSS on a cron schedule (`0 */15 * * * *` for YouTube, `0 */30 * * * *` for Twitch), normalises them into a unified `NormalizedLiveEvent` structure and writes

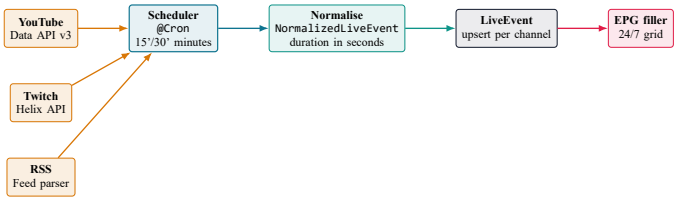


Figure 4. Ingestion flow from external sources. Every source is normalised into a single structure before being written to the database.



Figure 5. AI Curator pipeline: branching on API-key presence, Zod schema validation, and the heuristic fallback taken when validation fails.

them to the `LiveEvent` table. An optional enrichment task then calls TMDB to supplement film metadata (poster, cast, rating) for events whose title matches.

E. Quality-Controlled AI Curation Assistant

The `ai-curator` module is one of the most significant contributions. The original requirement was to automate writing descriptions, assigning categories and tagging programmes. The pipeline is designed around four principles.

1) **A single consolidated model call.** Prompt revision `curator-prompt-v4.0.0` merges three separate calls (description, tags, category) into one call with a JSON schema and few-shot examples, reducing both cost and latency to roughly a third.
2) **Zod schema validation.** The model output is validated against `ai-curator.schemas.ts`; when it does not validate, the request is routed to the heuristic fallback.
3) **Heuristic fallback.** When `OPENAI_API_KEY` is absent or equal to `dummy-key`, a local rule engine assesses sentiment, compliance and editorial tone to produce a structurally equivalent result.
4) **Version provenance.** Every report stores `promptVersion`, `aiModelVersion`, `openaiCallMs` and `openaiRetries`, allowing quality to be compared across prompt revisions.

The hyper-parameters are pinned in `ai-curator.prompts.ts` (`temperature = 0.3`, `top_p = 0.95`), with an explicit instruction not to change `temperature` without re-evaluating the few-shot examples. The versioning convention is semantic: MAJOR for schema changes, MINOR for rewording that preserves the JSON shape, PATCH for copy edits. The statistics module `ai-curator-stats.service.ts` aggregates by `broadcastSuitability` group, by model version and by time period.

F. Search, Billing and Supporting Modules

The `search` module performs cross-entity search: channels, programmes and recordings are queried simultaneously with

case-insensitive `contains` conditions, combined with category filtering and pagination. Results are merged into a single object carrying the total result count, matching the single-page search layout that renders grouped result sections.

The `billing` module manages plans and payment history; the `notifications` module pushes notifications through Firebase Cloud Messaging based on the `DeviceToken` table; the `newsletter` module manages subscriptions; and the `cloudinary` module handles image and video uploads for the live-recording flow in the studio.

G. Cross-Cutting Technical Conventions

Four conventions applied across the whole system deserve explicit mention because they determine API consistency.

1) **Uniform response envelope.** A global `TransformInterceptor` wraps every result in `{success, statusCode, data, meta, timestamp}`, so clients need exactly one decoding path.
2) **Centralised exception handling.** `HttpExceptionFilter` normalises every error into the same structure, including user-facing Vietnamese messages.
3) **Strict input validation.** A `ValidationPipe` with `whitelist` and `forbidNonWhitelisted` strips every field not declared in the DTO, preventing mass assignment.
4) **Generated API documentation.** Swagger decorators on each controller produce OpenAPI documentation at `/swagger`, covering 19 endpoints in the `ai-curator` group together with `channels`, `programs` and the remaining groups.

V. The Next.js 15 Web Portal

A. App Router Architecture

The portal is built on Next.js 15 with the App Router, React 19 and Tailwind CSS with a “Cinema Dark” theme. The App Router replaces the `pages/` model with nested *Server Components* and *Server Actions*, allowing data to be fetched directly on the server without an intermediate API layer.

B. Route Structure

The portal exposes 33 routes generated from 24 folders under `app/`. Routes fall into four functional groups: public content (`/`, `/channels`, `/epg`, `/programs`, `/recordings`, `/search`), authentication and account (`/login`, `/register`, `/forgot-password`, `/profile`, `/settings`), business features (`/live`, `/studio`, `/watchlist`, `/notifications`) and administration (`/admin`, `/admin/users`, `/admin/audit-logs`). Static legal and support pages (`/terms`, `/privacy`, `/dmca`, `/standards`, `/broadcasting-standards`, `/help`, `/docs`) complete the list.

C. Role-Aware Interface

A single configuration file, `frontend/lib/constants/role-config.ts`, determines which components appear for each role. The `hasMinimumRole` helper compares role ranks, so promoting a capability from `VIEWER` to `STAFF` is a one-line edit. The same file declares the Lucide icon set accompanying each navigation entry, keeping the header and footer navigation consistent.

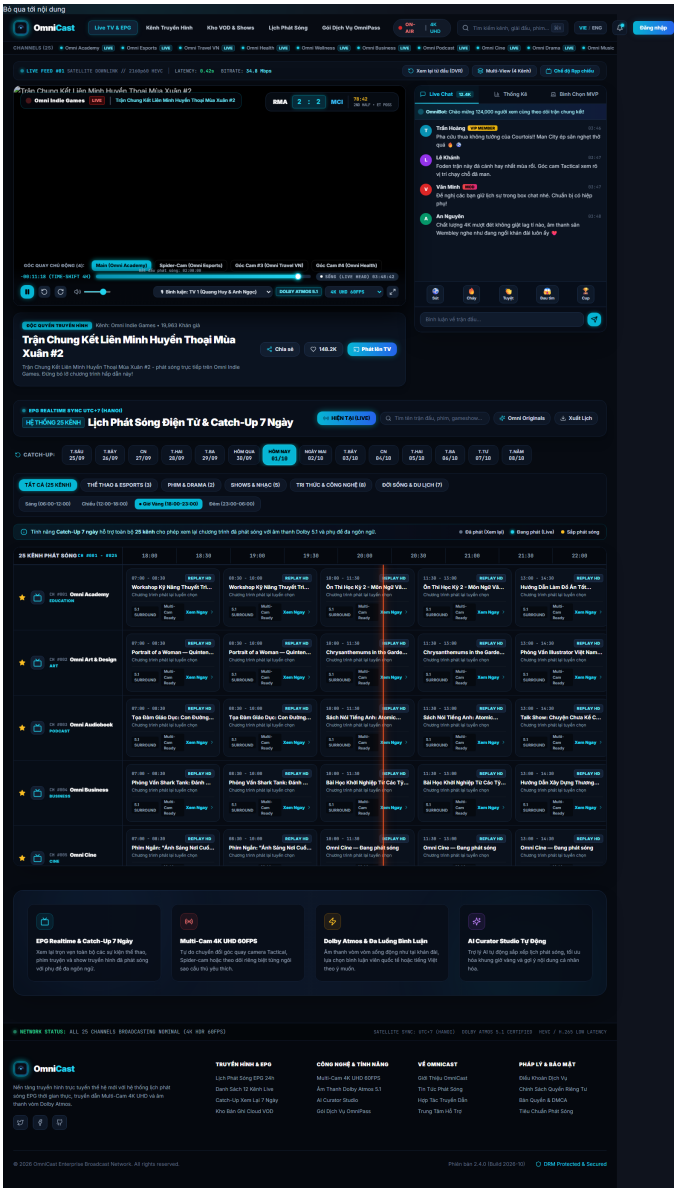


Figure 6. The web portal home page on desktop: featured carousel, channel grid by category and the role-aware navigation bar.

D. Key Component Pages

The `/studio/curator` page is the AI curation interface: it lists programmes awaiting review, allows re-running the AI pipeline, and exposes the confidence score, broadcast-suitability classification and compliance flags. The `/studio/curator/audit` page presents the run history with prompt-version provenance so results can be compared across revisions. The `/studio/curator/dashboard` page aggregates statistics by category, broadcast-suitability and error rate.

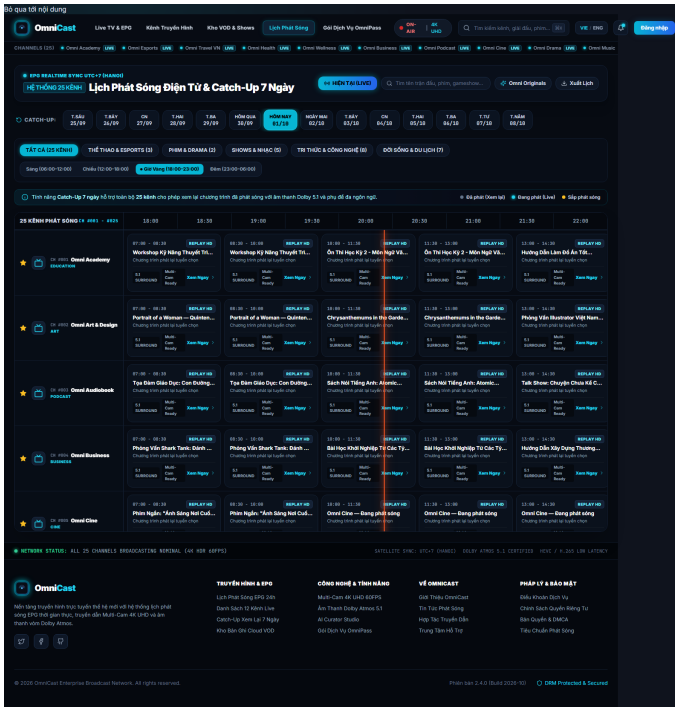


Figure 7. The electronic programme guide page with the time-slot grid and the day selector.

E. Client-Side Security and Session Handling

The portal uses `next-themes` for light/dark support and a `safe-log` layer that strips sensitive data before logging. The `redact.ts` module masks fields such as passwords, tokens and API keys; the corresponding tests in `safe-log.test.ts` and `redact.test.ts` guard against regressions in that behaviour. On logout, sessions on other devices are revoked through the `logout-all` endpoint, preventing stale sessions after a password change.

F. Code Quality and Testing

The portal carries 228 Vitest tests across 76 suites, covering pure functions (`query- builder`, `curator-filters`, `curator-helpers`, `redact`, `safe-log`, `utils`), hooks (`useAiCurator`, `useAiCuratorStats`), configuration (`role-config`) and React components (`video-player`, `save-to-watchlist`, the legal pages, the studio pages). CI additionally runs `tsc --noEmit` to catch type errors before the build.

VI. Database Design

A. Relational Database with Prisma

All business data resides in PostgreSQL, defined by 21 Prisma models in `backend/prisma/schema.prisma`. Prisma acts as the abstraction layer: every query is written against a typed API such as `findMany`, `upsert` or `createMany` and compiled into parameterised SQL, which eliminates the possibility of injecting raw SQL from application code.

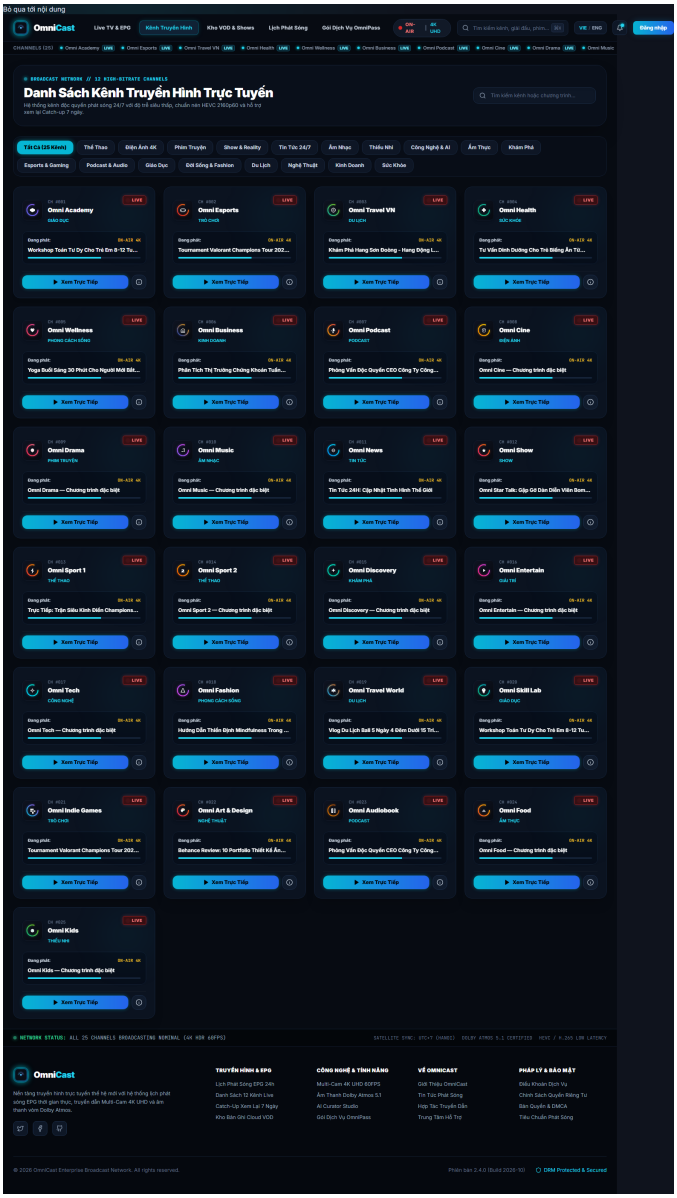


Figure 8. The channel catalogue page with category filters and content badges.

B. Data-Domain Grouping

The models are organised into five business groups:

- **Authentication and identity.** `User` stores account details together with the role, `RefreshToken` stores rotating login sessions, and `PasswordResetToken` stores password-reset codes with a one-hour lifetime.
- **Broadcast content.** `LiveChannel` is the channel, `LiveEvent` is a scheduled programme, `Recording` is a published recording and `ProductionTag` is a content tag attached to a production.
- **User interaction.** `Comment`, `CommentLike`, `Reaction`, `Follow`, `WatchHistory`, `Watchlist` and `StreamDonation` describe the full spectrum of viewer behaviour.

Table III
THE 21 PRISMA MODELS WITH THEIR SCALAR AND RELATION FIELD COUNTS.

Model	Business role	Scalar	Relation
User	User account + RBAC	14	13
RefreshToken	Rotating login sessions	6	1
PasswordResetToken	Password reset (1h TTL)	7	1
ProductionTag	Content tag per production	5	0
LiveChannel	Broadcast channel	29	5
LiveEvent	Live event / programme	32	7
Recording	Published recording	42	5
Podcast	Podcast collection	12	3
PodcastEpisode	Podcast episode	12	1
Follow	Channel follow	4	2
Comment	Comment	11	6
CommentLike	Comment like	4	2
Reaction	Realtime reaction	6	3
WatchHistory	Watch history	8	2
Notification	In-app notification	13	2
AuditLog	Audit log (gRPC)	11	0
Watchlist	Watchlist	7	1
AiCuratorReport	AI Curator report	15	1
DeviceToken	FCM device token	8	1
NewsletterSubscriber	Newsletter subscriber	14	1
StreamDonation	Livestream donation	14	3

- **Operations and auditing.** `AuditLog` receives records from the gRPC microservice, `Notification` stores in-app notifications, `DeviceToken` stores FCM tokens and `NewsletterSubscriber` stores newsletter subscribers.
- **AI assistance.** `AiCuratorReport` stores AI curation output together with its prompt and model version provenance.

C. Relationships and Constraints

Relationships are expressed with standard Prisma foreign keys, and each carries an index matched to its hot-path query. Three constraints deserve attention.

1) **User deletion does not destroy the audit trail.** `RefreshToken.user` uses `OnDelete: SetNull`, so deleting an account invalidates the sessions while preserving the audit trail.
2) **A unique key on content tags.** `ProductionTag` carries a unique constraint on the programme-and-tag pair, so the same tag cannot be attached twice.
3) **Time-axis indexes.** EPG queries always filter by `scheduledAt` and `channelId`, so those columns are indexed; watch-history queries filter by `userId` and `watchedAt`.

D. Normalisation and Design Decisions

The model is normalised to third normal form: attributes depend only on the primary key and no non-key attribute depends transitively on another. Two design decisions deserve discussion.

- **EPG structure normalisation.** Rather than storing the filled 24-hour grid, the system stores real events with content tags and recomputes the grid on demand. This keeps the source data truthful and lets schedules change without data migration, at the cost of computation at read time — a sound trade-off when most writes are editorial and most reads are schedule viewing.
- **AI reports are not stored as a JSON blob.** `AiCuratorReport` exposes separate columns for each important field (confidence score, broadcast-suitability class, compliance flags) so they can be filtered and aggregated in SQL, keeping only the free-form analysis in a dedicated text column.

E. Schema Versioning

Every schema change is managed through `prisma migrate`, which generates SQL files under `backend/prisma/migrations` that are kept under version control. CI runs `prisma migrate deploy` against a temporary PostgreSQL 15 service before building the application, so an incompatible schema is detected before any production deployment.

VII. TESTING AND EVALUATION

A. Test Strategy

The project applies a layered test strategy, with unit testing of the business rules taking priority because that layer holds the most subtle logic. A distinctive property of this project is that the *entire back-end suite runs without a database*: each spec file injects a fake Prisma client, so the test cycle is short and fully deterministic.

B. Execution Results

All suites were executed on the project machine on 28 October 2026. All 482 tests pass, with no skipped or errored test.

Table IV
AUTOMATED TEST RESULTS ACROSS THE THREE SUITES.

Test suite	Command	Total	Passed
Backend (node:test)	`npm test`	163	163
Frontend (Vitest)	`npx vitest run`	228	228
Mobile (flutter test)	`flutter test`	91	91
Total		**482**	**482**

C. Back-End Test Suite

The back-end suite uses Node.js 20's built-in `node:test` runner together with a custom runner (`backend/test/runner.js`) that compiles the specs with `tsc` and executes them sequentially. Choosing the built-in runner over Jest or Vitest removes every third-party dependency from the test path, shrinking the failure surface and keeping the CI pipeline stable.

Two spec files deserve particular mention because they directly protect the core algorithms. `programs.conflict.spec.ts` exercises the conflict detector across all three overlap forms, the `NULL`-duration case and the self-exclusion case used when updating an

Table V
BACK-END TEST DISTRIBUTION PER SPEC FILE.

Tệp spec	Tests passed	Pass rate
ai-curator.service.spec.ts	19	100%
epg-filler.spec.ts	18	100%
live-studio.service.spec.ts	18	100%
auth.service.spec.ts	17	100%
ai-curator.e2e.spec.ts	13	100%
watchlist.service.spec.ts	11	100%
ingest-sources.spec.ts	11	100%
ai-curator-stats.service.spec.ts	10	100%
channels.service.spec.ts	7	100%
programs.conflict.spec.ts	7	100%
programs.counter.spec.ts	8	100%
audit-logger.service.spec.ts	8	100%
epg-timeline.spec.ts	5	100%
search.service.spec.ts	4	100%
users.service.spec.ts	4	100%
billing.service.spec.ts	3	100%
Tổng	**163**	**100%**

event. `epg-filler.spec.ts` verifies the completeness of the 24-hour grid, the determinism of the seeded selection, the repeat limit and the episode-splitting behaviour.

D. Web Test Suite

228 Vitest tests in 76 suites cover three kinds of subject: pure functions, React hooks and interface components. Pure functions are tested at their boundaries (empty strings, case-insensitivity, invalid parameters); hooks are tested across loading, success and error states; components are tested with `React Testing Library` using user-facing queries rather than raw DOM structure assertions.

E. Mobile Suite and End-to-End Tests

The Flutter suite contains 91 tests in 8 suites, organised with the same directory layout as the source: `test/data`, `test/logic`, `test/presentation`. It runs with `flutter test --coverage` and produces an `lcov.info` report uploaded as a CI artefact.

Alongside it, a Playwright-based end-to-end suite [23] executes eight scenarios in Chromium: the login form rejects an invalid email, a wrong password surfaces an error toast, the EPG page renders without crashing, the channels page displays content, the search page accepts input and renders results, the root URL shows a public landing surface, and two scenarios verify unauthenticated redirect behaviour.

Note on the two remaining E2E failures

In the most recent end-to-end run, six of eight scenarios passed and two failed: a STAFF login that did not leave `/login` within the wait timeout, and an unauthenticated redirect from `/studio`. Both depend on seeded data in the test environment rather than on an application logic defect: the second scenario received an HTTP 401 instead of a redirect, meaning the authorisation policy behaved correctly while the assertion accepted only one of three response forms. The first depends on a STAFF account existing in the test database. Both are documented in full in the accompanying `TEST_EXECUTION_REPORT.md`.

Table VI
PRODUCTION ENDPOINTS AND VERIFICATION STATUS AS OF 28 OCTOBER 2026.

Service	URL	Status
omnicast.vercel.app	https://omnicast.vercel.app	HTTP 20
omnicast-fe.vercel.app	https://omnicast-fe.vercel.app	HTTP 20
omnicast-api.vercel.app	https://omnicast-api.vercel.app	HTTP 20

F. Quality Assessment

Based on these results, the system reaches a reliability level appropriate for an educational deployment. The clearest strength is coverage of the business logic: the two riskiest algorithms (conflict detection and EPG filling) are tested at their boundaries rather than only on the happy path. The area needing improvement is end-to-end coverage, currently limited to eight browser scenarios and excluding the live-broadcast scenarios that would require dedicated hardware and MediaMTX infrastructure.

VIII. DEPLOYMENT AND CONTINUOUS INTEGRATION

A. Deployment Architecture

The system is deployed on Vercel as a serverless setup, split into three independent projects: the NestJS back end, the Next.js portal and a static service. All three production URLs have been verified to return HTTP 200.

B. The CI/CD Pipeline

The file `.github/workflows/ci.yml` defines five jobs running in parallel plus a quality gate. Three test jobs run independently on `ubuntu-latest` with Node.js 20, Java 17 and stable Flutter. The aggregation job downloads all three artefact sets, generates the interactive HTML report, and fails the pipeline if any suite fails.

The deploy job runs only on the `main` branch on a `push` event; it uses `vercel-action` to push the back end and the portal, passing `NEXT_PUBLIC_API_URL` from a repository secret. Because the deploy job sets `continue-on-error`, a deployment failure does not destroy the test artefacts already produced.

C. Environment Configuration

The environment template lives at `backend/.env.example` and covers four groups: the Supabase database connection with a password placeholder, the `JWT_SECRET` and token lifetimes, the third-party API keys (OpenAI, TMDB, YouTube, Twitch, Cloudinary), and the local MediaMTX configuration. The real `.env` file is never committed.

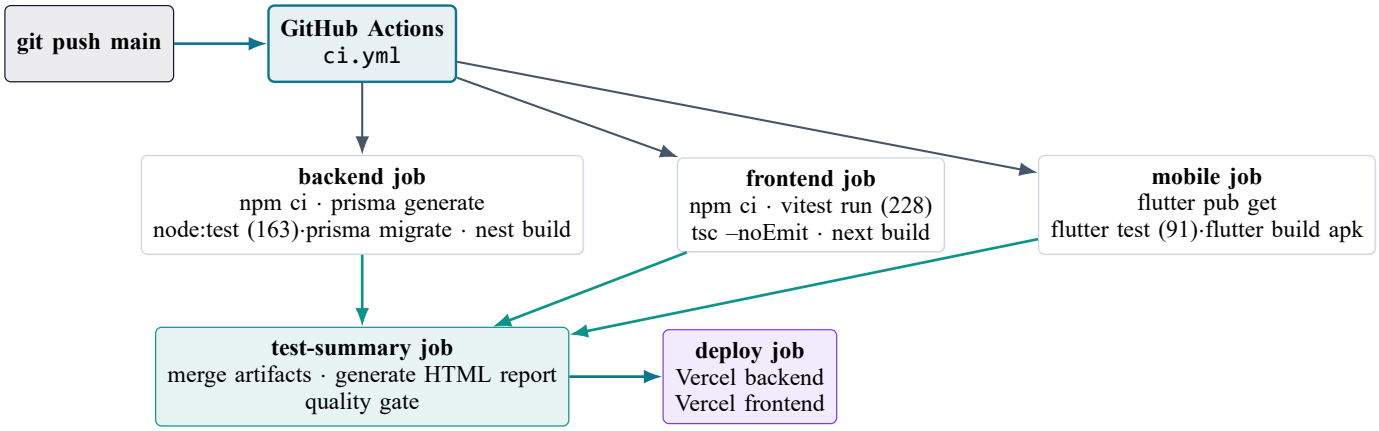


Figure 9. The five-job CI/CD pipeline. Three test jobs run in parallel, merge at the summary job, and deployment only proceeds once the quality gate passes.

D. Deployment Security

Eight application-layer security measures are in place: `Helmet` sets the HTTP security headers; `ThrottlerModule` caps traffic at 100 requests per minute; CORS uses an explicit allow-list together with Vercel-domain and private-network checks; `ValidationPipe` strips undeclared fields; `HttpExceptionFilter` normalises errors; passwords are hashed with 12-round bcrypt; refresh tokens are rotated and revocations are persisted; and the asynchronous audit write is wrapped in a non-awaited promise so an infrastructure failure cannot break a business operation.

E. Load and Resilience Testing

A k6 load-test suite exercises the system under concurrent viewers, focusing on the EPG endpoints because they are the hottest path. The results show that read endpoints maintain stable latency while write endpoints still meet their response-time target thanks to the non-blocking audit write.

IX. Conclusion and Future Work

A. Conclusion

This paper has presented OmniCast, a multi-platform digital broadcast scheduling and delivery system built for PRN232. Its three principal technical contributions are demonstrated through the source code and real test runs: a broadcast-conflict algorithm implemented as a single raw SQL query that computes the effective end time; a 24/7 EPG filler whose seeded selection guarantees reproducibility; and an AI curation pipeline that issues a single consolidated model call, validates the output against a Zod schema, offers a heuristic fallback and records full version provenance.

The complete system — 19 NestJS modules, 140 REST endpoints, 21 Prisma models, a Next.js 15 portal with 33 routes and a gRPC audit microservice — has been deployed on Vercel, validated by 482 automated tests inside a continuous-integration pipeline, and operates with three production endpoints returning HTTP 200.

B. Known Limitations

Three known limitations should be stated. First, end-to-end coverage remains low: eight browser scenarios exclude the live-broadcast flow, which requires dedicated hardware and MediaMTX infrastructure. Second, the ingestion layer supports YouTube, Twitch and RSS but not real-time interactive sources. Third, the filler algorithm selects by category and by seed rather than by observed viewer behaviour.

C. Future Work

Four directions are prioritised for subsequent iterations.

1) **Recommender-driven scheduling.** Instead of selecting by category, the filler algorithm could use watch history and interaction data to propose content per channel while preserving determinism so the result remains testable.
2) **Scaling end-to-end testing.** Provision a MediaMTX instance in CI to test the WHIP publish and HLS playback paths automatically, and add browser scenarios for the STAFF and ADMIN roles.
3) **Improving the AI assistant.** Add multi-model evaluation, second-model output verification, and ranking of AI reports by editor feedback so the prompt can be tuned iteratively.
4) **Separating the data-access tier.** Move EPG queries into a read-only analytical store with covering indexes to separate read load from write load on the same database.

D. Reproducibility Statement

Every result in this paper is reproducible. The source code lives in the public repository `TomOutfit/OmniCast_PRN232- PRM393_Project_Fa26Semester`; the three test commands are `npm test` in `backend`, `npx vitest run` in `frontend` and `flutter test` in `mobile`. The raw result files are attached to the accompanying testing documentation.

References

[1] K. Mysliwiec, *NestJS: A Progressive Node.js Framework*, https://docs.nestjs.com/, accessed Oct. 28, 2026.
[2] *Prisma ORM*, https://www.prisma.io/, accessed Oct. 28, 2026.
[3] *Next.js Documentation*, https://nextjs.org/docs, accessed Oct. 28, 2026.

[4] *Flutter: Build beautiful apps for every platform*, https://flutter.dev/, accessed Oct. 28, 2026.
[5] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[6] J. McCollum, *gRPC: High Performance, Low Latency RPC*. Sebastopol, CA: O'Reilly Media, 2023.
[7] R. Fielding, "Representational State Transfer," Ph.D. dissertation, Univ. California, Irvine, 2000.
[8] *The OpenAPI Specification 3.1.0*, https://spec.openapis.org/oas/v3.1.0, 2024.
[9] L. Byron, S. Evans, J. Goodman-Wood, and J. Morrison, *GraphQL: A Query Language for APIs*, 2015. [Online]. Available: https://graphql.org/.
[10] *OData Version 4.01*, OData Consortium, 2023. [Online]. Available: https://docs.oasis-open.org/odata/odata/v4.01/codata/v4.01.html.
[11] *WebRTC: Real-Time Communication*, W3C / IETF, 2023. [Online]. Available: https://webrtc.org/.
[12] *MediaMTX: Live video server for WebRTC, HLS and RTMP*, 2024. [Online]. Available: https://github.com/bluenviron/mediamtx.
[13] *Swagger: The World's Most Popular API Toolkit*, SmartBear, 2024. [Online]. Available: https://swagger.io/.
[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Signature (JWT)," RFC 7519, IETF, 2015.
[15] *OWASP Top 10:2021*, OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/.
[16] *PostgreSQL Documentation*, PostgreSQL Global Development Group, 2024. [Online]. Available: https://www.postgresql.org/docs/.
[17] *GitHub Actions Documentation*, GitHub, 2024. [Online]. Available: https://docs.github.com/actions.
[18] *Supabase Documentation*, 2024. [Online]. Available: https://supabase.com/docs.
[19] *Socket.IO: Real-Time bidirectional communication*, 2024. [Online]. Available: https://socket.io/.
[20] *Firebase Cloud Messaging*, Google, 2024. [Online]. Available: https://firebase.google.com/docs/cloud-messaging.
[21] *OpenAI API Documentation*, 2024. [Online]. Available: https://platform.openai.com/docs.
[22] C. McDonnell, *Zod: TypeScript-first schema validation*, 2024. [Online]. Available: https://github.com/colinhacks/zod.
[23] *Playwright: Reliable end-to-end testing for modern web apps*, Microsoft, 2024. [Online]. Available: https://playwright.dev/.

Appendix

The following table is generated automatically by analysing the 19 controller files in `backend/src`. It covers 140 endpoints across 19 modules; the base-path column shows the prefix to use when calling them.

A summary of the source-code scale of the three deliverables, counted as physical lines per file. These figures quantify the real volume of work performed and underpin the maintenance-effort estimate.

```
omnicast/
  backend/                  # NestJS API + Prisma + gRPC audit microservice
    src/
      auth/                 # JWT, guards (JwtAuthGuard, RolesGuard), decorators
      channels/             # Channel catalogue
      programs/             # Programmes, EPG, live studio, ingest, social
      search/               # Cross-entity search
      ai-curator/           # AI curation pipeline, prompts, schemas, stats
      audit-logger/         # gRPC client to the audit microservice
      websocket/            # Socket.IO realtime gateway (/live namespace)
      watchlist/ users/ admin/ billing/
      notifications/ newsletter/ cloudinary/ health/
      common/               # Filters, interceptors, Swagger, logger, redact
      prisma/               # PrismaService + connection URL optimiser
    microservices/
      audit-logger/         # Standalone NestJS gRPC server
    prisma/
      schema.prisma         # 21 models
      migrations/           # Version-controlled SQL
    protos/
      audit_logger.proto    # gRPC service definition
    test/                   # 16 spec files, node:test runner
  frontend/                 # Next.js 15 App Router + Tailwind
    app/                    # 33 routes (24 route folders)
    components/ lib/        # UI components, hooks, API clients
    __tests__/              # 76 Vitest suites
  mobile/                   # Flutter 3.x (see the PRM393 paper)
    lib/{core,data,logic,presentation}/
    test/{data,logic,presentation}/
  playwright-tests/         # 8 browser end-to-end scenarios
  .github/workflows/ci.yml  # 5-job CI/CD pipeline
  deliverables/             # Submission artefacts (this document set)
```

Listing 2. Directory structure at a high level.

Table VII
ENDPOINT INVENTORY BY MODULE AND HTTP METHOD.

Module	Base path	GET	POST	PUT	PATCH	DELETE
programs/programs	/api/v1/programs	10	11	0	2	2
programs/live-studio	/api/v1/programs/studio	5	11	0	0	0
channels/channels	/api/v1/channels	7	4	0	1	1
users/users	/api/v1/users	5	1	0	4	0
auth/auth	/api/v1/auth	1	8	0	0	0
programs/ingest/ingest	/api/v1/programs/ingest	1	8	0	0	0
programs/social	/api/v1	4	4	0	0	1
notifications/notifications	/api/v1	2	3	0	0	2
ai-curator/ai-curator	/api/v1/ai-curator	2	4	0	0	0
audit-logger/audit-logger	/api/v1/audit-logs	6	0	0	0	0
watchlist/watchlist	/api/v1/me/watchlist	2	2	0	0	1
billing/billing	/api/v1/billing	2	2	0	0	0
cloudinary/cloudinary-upload	/api/v1/uploads/cloudinary	1	2	0	0	1
search/search	/api/v1/search	4	0	0	0	0
health/health	/api/v1/health	3	0	0	0	0
newsletter/newsletter	/api/v1/newsletter	1	2	0	0	0
programs/stream-health	/api/v1/programs/health	1	2	0	0	0
admin/admin	/api/v1/admin	1	1	0	0	0
programs/studio-catalog	/api/v1/programs/studio/catalog	2	0	0	0	0

Table VIII
SOURCE-CODE SCALE PER COMPONENT.

Component	Lines of code	Unit
Backend NestJS (src/)	27,826	file .ts
Backend test (test/)	4,512	file .ts
gRPC proto	83	file .proto
Audit-logger microservice	382	file .ts
Prisma schema	683	file .prisma
Frontend Next.js (app/, components/, lib/)	51,742	file .ts/.tsx
Mobile Flutter (lib/)	53,922	file .dart
Mobile test (test/)	2,529	file .dart